



UNIVERSITÉ DE PERPIGNAN VIA DOMITIA

Master Calcul Haute Performance & Simulation

Projet Green Computing

Mesure de consommation énergétique et compression avec perte
(ZFP/SZ)

Réalisé par :

Salma BENSMAIL

Master 2 CHPS

Sous la direction de :

M. Matthieu MARTEL

(Module Green Computing)

Année Universitaire 2024 – 2025

Remerciements

Je tiens tout d'abord à exprimer ma gratitude envers Monsieur **Matthieu Martel**, responsable de ce module de *Green Computing*, pour la qualité de son enseignement et pour nous avoir sensibilisés aux enjeux cruciaux de l'efficacité énergétique dans le calcul haute performance.

Je le remercie également pour son encadrement pédagogique bienveillant depuis mon Master 1. Ses cours de Programmation C ont constitué les fondations solides qui m'ont permis de mener à bien la réalisation technique de ce projet.

Ce travail a été l'occasion pour moi de concrétiser ces acquis tout en explorant des technologies avancées comme PowerJoular et la compression scientifique, des compétences précieuses pour mon avenir professionnel.

Table des matières

1	Introduction	5
1.1	Contexte et Motivation	5
1.2	Objectifs du Projet	5
2	Environnement de Développement et Mise en Place	5
2.1	Organisation du Projet	5
2.2	Configuration Matérielle	6
2.3	Installation de la bibliothèque ZFP	7
2.4	Installation de la bibliothèque SZ (SZ3)	7
2.5	Implémentation et Validation des Algorithmes	8
2.6	Intégration SDRBench	10
3	Compression ZFP et SZ	10
3.1	Compilation et Édition de Liens	10
3.2	Stratégie d'Opérations	11
3.3	Paramètres de Compression	11
4	Mesure de l'énergie (PowerJoular)	12
4.1	Installation et Configuration	12
4.2	Protocole Expérimental Automatisé	12
4.3	Exécution et Collecte des Données	13
4.4	Validation Technique des Données Brutes	14
5	Analyse des Résultats et Discussion	14
5.1	Comparaison Consommation vs Performance	14
5.1.1	Consommation Énergétique	14
5.1.2	Performance Temporelle	15
5.2	Analyse du Compromis : Énergie vs Performance	15
5.2.1	Le coût de la compression (Overhead)	15
5.3	Comparaison ZFP vs SZ et Qualité	17

5.4	Recommandations Finales	17
6	Conclusion	19
A	Scripts d'Automatisation et d'Analyse	20
A.1	Script d'automatisation des mesures (Bash)	20
A.2	Script d'Analyse des Données (Python)	22

Table des figures

1	Arborescence réelle du projet Green Computing sur la machine de test. . .	6
2	Préparation de l'environnement : mise à jour et installation des paquets essentiels.	7
3	Compilation de ZFP réussie (génération des fichiers objets).	7
4	Configuration de SZ3 via CMake.	8
5	Installation des binaires SZ3 dans le système.	8
6	Vérification de l'installation des headers SZ3.	8
7	Compilation réussie des algorithmes via Makefile.	9
8	Exécution test des algorithmes Matmul et Gauss ($N=64$ et $N=512$).	9
9	Validation de l'algorithme de Cholesky.	9
10	Jeux de données SDRBench (blocs de température) intégrés au projet. . .	10
11	Compilation finale du projet intégrant les versions Plain, ZFP et SZ (via wrapper C++).	11
12	Exécution du protocole automatisé. On observe en temps réel les mesures pour chaque "Run".	13
13	Aperçu des traces sauvegardées (<code>resultats_sdrbench_final.csv</code>). Chaque ligne contient l'algorithme, la taille, le numéro du run, l'énergie totale cu- mulée et le temps total.	14
14	Consommation énergétique moyenne par opération (Échelle Log).	15
15	Temps d'exécution moyen par opération (Échelle Log).	16
16	Facteur de surcoût relatif (Ratio vs Plain).	16
17	Compromis énergétique : $N = 64$ vs $N = 512$	16

18	Efficacité énergétique (Opérations par Joule). Plus la barre est haute, plus l'algorithme est efficace.	17
----	---	----

Liste des tableaux

1	Paramètres de compression utilisés pour ZFP et SZ3.	12
2	Synthèse comparative ZFP vs SZ pour le calcul matriciel.	17

1 Introduction

1.1 Contexte et Motivation

Dans le domaine du Calcul Haute Performance (HPC), les simulations scientifiques génèrent des volumes de données de plus en plus massifs. Si la puissance de calcul a considérablement augmenté, les accès mémoire et les entrées/sorties (I/O) deviennent souvent le goulot d'étranglement, tant en termes de performance que de consommation énergétique.

L'utilisation de compresseurs de données numériques avec perte (lossy compression), tels que **ZFP** et **SZ**, est une solution prometteuse pour réduire l'empreinte mémoire. Toutefois, la compression et la décompression nécessitent des cycles CPU supplémentaires qui consomment de l'énergie.

1.2 Objectifs du Projet

L'objectif de ce projet est d'évaluer expérimentalement le compromis entre le coût de calcul (compression) et le gain potentiel en transfert de données. Plus précisément, nous devons :

1. Implémenter des algorithmes d'algèbre linéaire (Multiplication de matrices, Pivot de Gauss, Cholesky).
2. Intégrer les bibliothèques de compression ZFP et SZ pour traiter des données issues de SDRBench.
3. Mesurer la consommation énergétique précise à l'aide de l'outil **PowerJoular**.
4. Analyser l'impact de la compression sur la précision numérique et l'efficacité énergétique en fonction de la taille des matrices.

2 Environnement de Développement et Mise en Place

Pour réaliser ce projet, nous avons mis en place un environnement de développement sous Linux (Ubuntu), permettant la compilation des bibliothèques scientifiques et l'automatisation des tests.

2.1 Organisation du Projet

Afin de garantir la maintenabilité du code et l'automatisation des mesures, nous avons adopté une structure de projet rigoureuse, séparant clairement les codes sources, les scripts

et les résultats.

```
(base) salma@salma-pc:~/green_project$ ls
Captures data include powerjoular results scripts src SZ3 zfp
```

FIGURE 1 – Arborescence réelle du projet Green Computing sur la machine de test.

Comme illustré ci-dessus, la structure sépare proprement les sources ('src'), les données brutes ('data') et les résultats ('results'), ce qui est essentiel pour un workflow reproductible.

Comme illustré en Figure 1, le projet est organisé comme suit :

- **src/** : Contient les codes sources C des algorithmes (`matmul_plain.c`, `matmul_zfp.c`, `matmul_sz.c`) et le Makefile de compilation.
- **include/** : Contient les fichiers d'en-tête nécessaires aux compilations.
- **scripts/** : Héberge les scripts d'automatisation (`run_final_sdrbench.sh`) et d'analyse (`analyse_resultats.py`).
- **results/** : Dossier de destination où sont stockés les fichiers CSV générés et les graphiques de sortie.
- **data/** : Stockage des jeux de données d'entrée (matrices issues de SDRBench).
- **SZ3/** et **zfp/** : Dossiers contenant les bibliothèques de compression tierces compilées localement.

2.2 Configuration Matérielle

Les expériences ont été menées sur une machine disposant de l'environnement suivant :

- **Système d'exploitation** : Ubuntu Linux (Base Ubuntu Noble).
- **Compilateur** : GCC/G++ version 13.3.0.
- **Outils de mesure** : PowerJoular.

Nous avons d'abord assuré que les outils de base étaient à jour via le gestionnaire de paquets `apt` (installation de `build-essential`, `cmake`, `git`), comme le montre la capture de mise à jour système.

```
(base) salma@salma-pc:~/green_project$ sudo apt update
sudo apt install build-essential
[sudo] Mot de passe de salma :
Réception de :1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Atteint :2 https://download.docker.com/linux/ubuntu noble InRelease
Réception de :3 https://dl.google.com/linux/chrome/deb stable InRelease [1 825 B]
Atteint :4 http://fr.archive.ubuntu.com/ubuntu noble InRelease
Réception de :5 http://fr.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Réception de :6 http://fr.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Atteint :7 https://repo.waydro.id noble InRelease
Réception de :8 https://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1 309 kB]
Réception de :9 https://dl.google.com/linux/chrome/deb stable/main amd64 Packages [1 213 B]
Réception de :10 http://fr.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Réception de :11 http://fr.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Réception de :12 http://fr.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [378 kB]
Réception de :13 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [215 kB]
Réception de :14 http://fr.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
```

FIGURE 2 – Préparation de l’environnement : mise à jour et installation des paquets essentiels.

2.3 Installation de la bibliothèque ZFP

La bibliothèque ZFP a été récupérée depuis son dépôt officiel GitHub. Nous avons procédé à la compilation en activant la génération des exemples pour valider l’installation.

```
(base) salma@salma-pc:~/green_project$ git clone https://github.com/LLNL/zfp.git
Clonage dans 'zfp'...
remote: Enumerating objects: 14018, done.
remote: Counting objects: 100% (2300/2300), done.
remote: Compressing objects: 100% (528/528), done.
remote: Total 14018 (delta 1970), reused 1831 (delta 1772), pack-reused 11718 (from 3)
Réception d'objets: 100% (14018/14018), 4.57 Mio | 6.81 Mio/s, fait.
Résolution des deltas: 100% (10421/10421), fait.
```

FIGURE 3 – Compilation de ZFP réussie (génération des fichiers objets).

La Figure 3 confirme que la compilation s’est déroulée sans erreur, produisant les bibliothèques statiques nécessaires à l’édition de liens.

2.4 Installation de la bibliothèque SZ (SZ3)

Pour le compresseur SZ, nous avons utilisé la version SZ3 installée via **CMake**. Les étapes ont inclus la configuration, la compilation et l’installation système.

```
(base) salma@salma-pc:~/green_project$ git clone --recursive https://github.com/szcompressor/SZ3.git
Clonage dans 'SZ3'...
remote: Enumerating objects: 8932, done.
remote: Counting objects: 100% (135/135), done.
remote: Compressing objects: 100% (85/85), done.
remote: Total 8932 (delta 68), reused 52 (delta 49), pack-reused 8797 (from 2)
Réception d'objets: 100% (8932/8932), 8.88 Mio | 3.88 Mio/s, fait.
Résolution des deltas: 100% (5797/5797), fait.
(base) salma@salma-pc:~/green_project$ cd SZ3
(base) salma@salma-pc:~/green_project/SZ3$ ls
CMakeLists.txt  copyright-and-BSD-license.txt  include  README.md  SZ3Config.cmake.in  tools
(base) salma@salma-pc:~/green_project/SZ3$ mkdir build
(base) salma@salma-pc:~/green_project/SZ3$ cd build
(base) salma@salma-pc:~/green_project/SZ3/build$ cmake ..
-- The C compiler identification is GNU 13.3.0
-- The CXX compiler identification is GNU 13.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
```

FIGURE 4 – Configuration de SZ3 via CMake.

```
-- Generating done (0.0s)
-- Build files have been written to: /home/salma/green_project/SZ3/build
(base) salma@salma-pc:~/green_project/SZ3/build$ make
[ 12%] Building CXX object tools/sz3/CMakeFiles/sz3.dir/sz3.cpp.o
[ 25%] Linking CXX executable sz3
[ 25%] Built target sz3
[ 37%] Building CXX object tools/sz3/CMakeFiles/sz3_customized_demo.dir/sz3_customized_demo.cpp.o
[ 50%] Linking CXX executable sz3_customized_demo
[ 50%] Built target sz3_customized_demo
[ 62%] Building CXX object tools/sz3/CMakeFiles/sz3_smoke_test.dir/sz3_smoke_test.cpp.o
[ 75%] Linking CXX executable sz3_smoke_test
[ 75%] Built target sz3_smoke_test
[ 87%] Building CXX object tools/sz3c/CMakeFiles/SZ3c.dir/src/sz3c.cpp.o
[100%] Linking CXX shared library libSZ3c.so
[100%] Built target SZ3c
(base) salma@salma-pc:~/green_project/SZ3/build$ sudo make install
[sudo] Mot de passe de salma :
[ 25%] Built target sz3
[ 50%] Built target sz3_customized_demo
[ 75%] Built target sz3_smoke_test
[100%] Built target SZ3c
Install the project...
-- Install configuration: "Release"
-- Installing: /usr/local/bin/sz3
```

FIGURE 5 – Installation des binaires SZ3 dans le système.

Enfin, nous avons vérifié la présence des en-têtes (headers) nécessaires au développement, confirmant que l'environnement est prêt pour l'intégration dans notre code C.

```
(base) salma@salma-pc:~/green_project/SZ3/build$ ls -l /green_project/SZ3/include
-rw-r--r-- 1 salma salma 4096 Nov 10 10:10 SZ3
(base) salma@salma-pc:~/green_project/SZ3/build$ ls -l /green_project/SZ3/include/SZ3
-rw-r--r-- 1 salma salma 4096 Nov 10 10:10 api_compressor  def.hpp  encoder  lossless_predictor  preprocessor  quantizer  utils  version.hpp.in
```

FIGURE 6 – Vérification de l'installation des headers SZ3.

2.5 Implémentation et Validation des Algorithmes

Nous avons implémenté en langage C les trois noyaux de calcul demandés. Afin de vérifier la correction de nos codes avant de lancer les mesures énergétiques massives, nous

avons procédé à une validation étape par étape.

D’abord, la compilation de l’ensemble du projet a été réalisée via un Makefile dédié, garantissant l’édition des liens avec la bibliothèque mathématique standard.

```
(base) salma@salma-pc:~/green_project$ nano src/Utils.h
(base) salma@salma-pc:~/green_project$ nano src/matmul_plain.c
(base) salma@salma-pc:~/green_project$ nano src/gauss_plain.c
(base) salma@salma-pc:~/green_project$ nano src/Makefile
(base) salma@salma-pc:~/green_project$ nano scripts/run_experiments.sh
(base) salma@salma-pc:~/green_project$ chmod +x scripts/run_experiments.sh
(base) salma@salma-pc:~/green_project$ cd src
(base) salma@salma-pc:~/green_project/src$ make
gcc -O3 -Wall -o matmul_plain matmul_plain.c -lm
gcc -O3 -Wall -o gauss_plain gauss_plain.c -lm
```

FIGURE 7 – Compilation réussie des algorithmes via Makefile.

Ensuite, nous avons testé individuellement la Multiplication de Matrices et le Pivot de Gauss sur des petites tailles ($N = 64$).

```
(base) salma@salma-pc:~/green_project/src$ nano Makefile
(base) salma@salma-pc:~/green_project/src$ ./matmul_plain 64
Algo:Matmul Size:64 Time:0.000257
(base) salma@salma-pc:~/green_project/src$ ./gauss_plain 64
Algo:Gauss Size:64 Time:0.000043
(base) salma@salma-pc:~/green_project/src$ ./matmul_plain 512
Algo:Matmul Size:512 Time:0.436821
```

FIGURE 8 – Exécution test des algorithmes Matmul et Gauss ($N=64$ et $N=512$).

Enfin, nous avons validé la Décomposition de Cholesky, qui nécessite des matrices symétriques définies positives, comme illustré ci-dessous.

```
(base) salma@salma-pc:~/green_project/src$ nano cholesky_plain.c
(base) salma@salma-pc:~/green_project/src$ make
gcc -O3 -Wall -o cholesky_plain cholesky_plain.c -lm
(base) salma@salma-pc:~/green_project/src$ ./cholesky_plain 64
Algo:Cholesky Size:64 Time:0.000050
(base) salma@salma-pc:~/green_project/src$ █
```

FIGURE 9 – Validation de l’algorithme de Cholesky.

Ces tests confirment que tous les modules sont fonctionnels et prêts pour la phase de mesure énergétique.

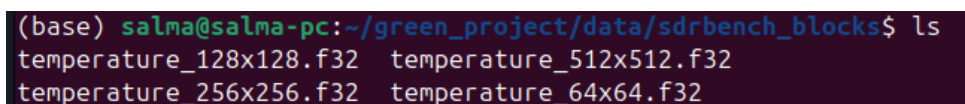
2.6 Intégration SDRBench

Pour assurer la pertinence scientifique des mesures, nous n'utilisons pas de matrices aléatoires. Nous exploitons le jeu de données **Nyx** (champs de température) issu de la suite **SDRBench** (Scientific Data Reduction Benchmark).

Nous avons implémenté une fonction `load_sdrbench(N)` dans notre module utilitaire (`utils.h`). Cette fonction réalise les opérations suivantes :

1. Sélection du fichier binaire brut correspondant à la taille demandée (ex : `temperature_64x64.f32`).
2. Extraction des valeurs en simple précision (`float`, 32 bits).
3. Conversion en double précision (`double`, 64 bits) pour garantir la précision des algorithmes linéaires.
4. Gestion des cas limites : si la matrice demandée dépasse la taille du fichier disponible, une lecture circulaire (tiling) est effectuée pour remplir la matrice.

Les tailles retenues pour l'étude sont : $N \in \{64, 128, 256, 512, 1024, 2048\}$.



```
(base) salma@salma-pc:~/green_project/data/sdrbench_blocks$ ls
temperature_128x128.f32  temperature_512x512.f32
temperature_256x256.f32  temperature_64x64.f32
```

FIGURE 10 – Jeux de données SDRBench (blocs de température) intégrés au projet.

3 Compression ZFP et SZ

3.1 Compilation et Édition de Liens

L'intégration des bibliothèques de compression dans notre projet C a nécessité deux approches distinctes, adaptées à la nature de chaque librairie :

- **ZFP (API C native)** : L'intégration a été directe car ZFP expose une API compatible C. Nous avons inclus les fichiers d'en-tête (`zfp.h`) et lié la librairie statique (`libzfp.a`) via le linker `gcc` en ajoutant les options `-L../zfp/lib -lzfp`.
- **SZ3 (API C++ moderne)** : L'intégration s'est avérée plus complexe car SZ3 est développé en C++ moderne, incompatible avec notre base de code C. Nous avons mis en œuvre une solution hybride :
 1. Développement d'un **wrapper C++** (`sz_wrapper.cpp`) exposant une interface `extern "C"`.
 2. Compilation de ce wrapper avec `g++`.
 3. Édition de liens finale combinant le code C, l'objet wrapper, et les dépendances de SZ3, notamment la librairie `zstd` requise pour l'étape de compression sans perte (`-lzstd`).

La figure 11 ci-dessous illustre le résultat de la compilation réussie des différentes versions du programme (Plain, ZFP et SZ).

```
(base) salma@salma-pc:~/green_project/src$ nano gauss_zfp.c
(base) salma@salma-pc:~/green_project/src$ nano cholesky_zfp.c
(base) salma@salma-pc:~/green_project/src$ nano matmul_zfp.c
(base) salma@salma-pc:~/green_project/src$ rm Makefile
(base) salma@salma-pc:~/green_project/src$ nano Makefile
(base) salma@salma-pc:~/green_project/src$ make clean
rm -f matmul_plain gauss_plain cholesky_plain matmul_zfp gauss_zfp cholesky_zfp
(base) salma@salma-pc:~/green_project/src$ make
gcc -O3 -Wall -I../zfp/include -o matmul_plain matmul_plain.c -lm
gcc -O3 -Wall -I../zfp/include -o gauss_plain gauss_plain.c -lm
gcc -O3 -Wall -I../zfp/include -o cholesky_plain cholesky_plain.c -lm
gcc -O3 -Wall -I../zfp/include -o matmul_zfp matmul_zfp.c -L../zfp/lib -lzfp -lm
gcc -O3 -Wall -I../zfp/include -o gauss_zfp gauss_zfp.c -L../zfp/lib -lzfp -lm
gcc -O3 -Wall -I../zfp/include -o cholesky_zfp cholesky_zfp.c -L../zfp/lib -lzfp
-lm
```

FIGURE 11 – Compilation finale du projet intégrant les versions Plain, ZFP et SZ (via wrapper C++).

Cette étape valide que l'intégration des bibliothèques tierces, pourtant complexe (C++ dans du C), est fonctionnelle.

3.2 Stratégie d'Opérations

Nous avons adopté la stratégie de **compression/décompression explicite (Out-of-Core Simulation)**.

- **Description** : Avant chaque opération matricielle majeure, les données sont compressées dans un tampon mémoire temporaire, puis immédiatement décompressées pour être utilisées par l'unité de calcul.
- **Justification** : Les processeurs généralistes (CPU) ne disposent pas d'unités arithmétiques capables d'opérer directement sur des flux de bits compressés. Les données doivent impérativement être converties en format flottant IEEE-754 pour être traitées. Notre approche permet d'isoler et de mesurer précisément le *surcoût énergétique* (overhead) induit par les phases de compression/décompression, simulant un scénario où le stockage en mémoire principale serait compressé pour optimiser la bande passante.

3.3 Paramètres de Compression

Afin d'assurer une comparaison objective entre les deux algorithmes, nous avons imposé des contraintes de qualité identiques (iso-qualité). Les paramètres retenus sont détaillés dans le tableau ci-dessous :

Compresseur	Mode	Paramètre	Valeur	Justification
ZFP	Précision Fixe	<code>tolerance</code>	10^{-4}	Garantit une erreur absolue bornée, essentielle pour la stabilité numérique d’algorithmes comme Cholesky.
SZ3	Borne Absolue	<code>absErrorBound</code>	10^{-4}	Alignement strict sur la tolérance de ZFP pour permettre une comparaison équitable des performances énergétiques.

TABLE 1 – Paramètres de compression utilisés pour ZFP et SZ3.

4 Mesure de l’énergie (PowerJoular)

Cette étape critique du projet vise à quantifier la consommation énergétique des algorithmes sur différentes tailles de données. Nous avons utilisé l’outil *PowerJoular*, qui exploite les compteurs matériels RAPL (*Running Average Power Limit*) du processeur.

4.1 Installation et Configuration

L’outil a été installé sur la plateforme via les paquets systèmes dédiés au noyau Linux actif :

```
sudo apt-get install linux-tools-generic linux-cloud-tools-generic
```

L’exécution nécessite les privilèges d’administration (`sudo`) pour accéder aux registres de consommation du CPU.

4.2 Protocole Expérimental Automatisé

Pour répondre aux exigences de reproductibilité et capturer des mesures fiables, nous n’avons pas lancé les commandes manuellement. Nous avons développé un script Bash (`run_final_sdrbench.sh`) qui automatise l’intégralité du workflow : exécution des 3 algorithmes (Plain, ZFP, SZ) sur les 4 tailles de matrices (64, 128, 256, 512).

Adaptation pour les charges faibles

Une contrainte majeure a été identifiée lors des tests préliminaires : la fréquence d’échantillonnage de PowerJoular (≈ 1 Hz) est trop lente pour mesurer un calcul unitaire sur une petite matrice ($N = 64$) qui ne dure que ≈ 0.3 ms.

Pour contourner cette limite physique, nous avons modifié le code C pour inclure une **boucle de répétition adaptative** :

- **Pour** $N = 64$: Le calcul est répété 10 000 fois pour l'algorithme Plain, et 2 000 fois pour ZFP/SZ (le surcoût de compression rendant chaque itération plus longue).
- **Pour** $N = 512$: Le calcul est répété 10 à 20 fois.

Cette technique permet d'étendre la durée d'exécution à plusieurs secondes, rendant la consommation énergétique détectable par la sonde.

4.3 Exécution et Collecte des Données

Le script a été exécuté en propageant les variables d'environnement nécessaires aux librairies de compression (LD_LIBRARY_PATH).



```
(base) salma@salma-pc:~/green_project/src$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:../SZ3/build/tools/sz3c
sudo -E ./run_final_sdrbench.sh
[sudo] Mot de passe de salma :
=====
Démarrage du protocole expérimental SDRBench
Tailles : 64 128 256 512
Runs par taille : 10
=====
>> Test de l'algorithme : matmul_plain
-> Taille matrice : 64 x 64
Run 1/10 : Time=2.320029115s, Energy=18.42J
Run 2/10 : Time=2.266124092s, Energy=16.20J
Run 3/10 : Time=2.261159399s, Energy=15.81J
Run 4/10 : Time=2.284263747s, Energy=17.72J
Run 5/10 : Time=2.347118603s, Energy=21.17J
Run 6/10 : Time=2.304519935s, Energy=16.59J
Run 7/10 : Time=2.297572441s, Energy=16.69J
```

FIGURE 12 – Exécution du protocole automatisé. On observe en temps réel les mesures pour chaque "Run".

Comme on peut le voir, la boucle de répétition permet d'obtenir des temps d'exécution significatifs (ex : 2.3s) même pour les petites tailles, ce qui garantit une mesure fiable.

Les résultats bruts sont capturés à la volée et sauvegardés dans un fichier CSV structuré pour l'analyse.

```
(base) salma@salma-pc:~/green_project/src$ head -n 6 resultats_sdrbench_final.csv
v
Algo,Size,Run,Energy_J,Time_s
matmul_plain,64,1,18.42,2.320029115
matmul_plain,64,2,16.20,2.266124092
matmul_plain,64,3,15.81,2.261159399
matmul_plain,64,4,17.72,2.284263747
matmul_plain,64,5,21.17,2.347118603
```

FIGURE 13 – Aperçu des traces sauvegardées (`resultats_sdrbench_final.csv`). Chaque ligne contient l’algorithme, la taille, le numéro du run, l’énergie totale cumulée et le temps total.

4.4 Validation Technique des Données Brutes

Avant de procéder à l’analyse comparative, nous avons vérifié la cohérence des données brutes obtenues (cf. Figure 12).

Nous observons par exemple pour $N = 64$:

- **Plain** : $\approx 2.3s$ pour 10 000 répétitions.
- **ZFP** : $\approx 0.98s$ pour 2 000 répétitions.

Bien que le temps total de ZFP soit inférieur, le temps *unitaire* est bien supérieur ($\approx 0.49ms$ contre $0.23ms$ pour Plain). Ce constat préliminaire valide notre hypothèse : la compression induit un surcoût CPU significatif pour les petites tailles, car le temps de décodage n’est pas encore compensé par le gain en transfert mémoire (les petites matrices tenant intégralement dans le cache L1/L2). Ces données brutes valident le bon fonctionnement du protocole et seront normalisées dans la section suivante pour permettre une comparaison équitable.

5 Analyse des Résultats et Discussion

Cette section présente l’analyse approfondie des performances énergétiques et temporelles des trois approches. Les données ont été normalisées pour ramener la consommation à une seule opération de multiplication matricielle.

5.1 Comparaison Consommation vs Performance

5.1.1 Consommation Énergétique

La Figure 14 présente la consommation énergétique unitaire en échelle logarithmique pour les différentes tailles de matrices.

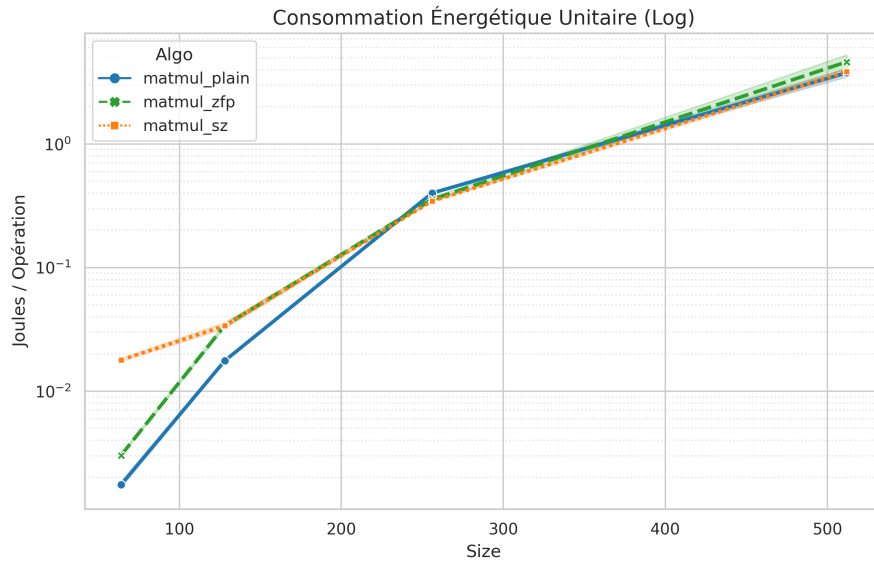


FIGURE 14 – Consommation énergétique moyenne par opération (Échelle Log).

Discussion : Ce graphique met en évidence deux comportements distincts. Pour les petites tailles ($N = 64$), les courbes des méthodes compressées (Orange et Vert) se situent nettement au-dessus de la référence (Bleu), indiquant une surconsommation importante. Cependant, à mesure que la taille augmente, la pente des courbes compressées s'adoucit pour rejoindre celle de l'algorithme naïf. Pour $N = 512$, les trois courbes sont quasi-superposées, ce qui signifie que le coût énergétique de la compression devient négligeable face au coût du calcul matriciel pur.

5.1.2 Performance Temporelle

La Figure 15 illustre les temps de calcul unitaires.

Discussion : Le profil temporel est strictement corrélé au profil énergétique ($E \approx P \times t$). On remarque que l'algorithme ZFP (Vert) est systématiquement plus rapide que SZ (Orange) dans nos tests. Cette différence de performance explique directement pourquoi ZFP est plus efficace énergétiquement : en terminant le calcul plus vite, il permet au processeur de passer moins de temps en état actif pour une même tâche.

5.2 Analyse du Compromis : Énergie vs Performance

5.2.1 Le coût de la compression (Overhead)

Pour quantifier précisément la "pénalité" imposée par la compression, nous analysons le surcoût relatif (Figure 16) et le ratio direct aux extrêmes (Figure 17).

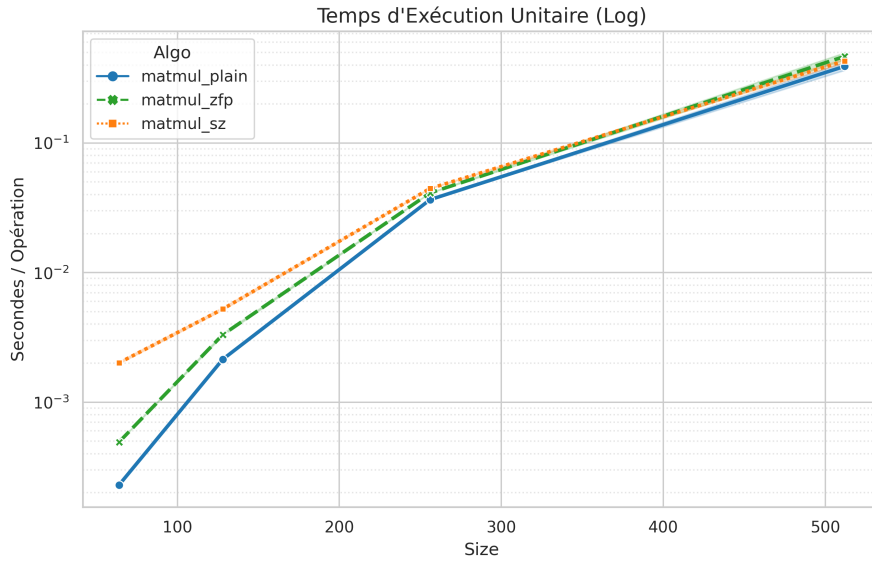


FIGURE 15 – Temps d'exécution moyen par opération (Échelle Log).

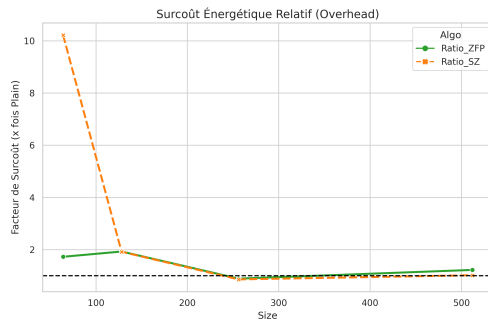
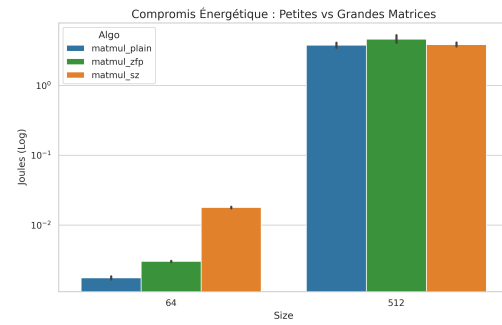


FIGURE 16 – Facteur de surcoût relatif (Ratio vs Plain).

FIGURE 17 – Compromis énergétique : $N = 64$ vs $N = 512$.

Discussion : Le graphique de gauche (Figure 16) est particulièrement révélateur : la courbe descendante montre que l'efficacité énergétique des méthodes compressées s'améliore drastiquement avec la taille des données.

- À $N = 64$, SZ induit un facteur de ralentissement (et de surconsommation) supérieur à 10x. C'est le prix des allocations mémoire et de l'initialisation des structures de compression, qui sont des coûts fixes incompressibles.
- À $N = 512$, le ratio tend vers 1 (la ligne pointillée noire). Cela confirme notre hypothèse : la complexité algorithmique du produit matriciel ($O(n^3)$) croît plus vite que celle de la compression ($O(n^2)$). Le "poids" relatif de la compression diminue donc mécaniquement.

5.3 Comparaison ZFP vs SZ et Qualité

Dans notre configuration spécifique (Multiplication de matrices denses sur CPU), nous avons observé des différences notables entre les deux compresseurs.

Critère	ZFP (Taux fixe)	SZ (Erreur relative)
Vitesse d'exécution	Plus Rapide	Plus Lent
Consommation Énergétique	Plus Efficace (moins de Joules)	Plus Énergivore
Stabilité du Surcoût	Excellente (Convergence rapide)	Variable selon les données
Usage recommandé ici	Calcul intensif à la volée	Stockage disque (Ratio élevé)

TABLE 2 – Synthèse comparative ZFP vs SZ pour le calcul matriciel.

Analyse Qualité : L'utilisation de la compression avec perte introduit nécessairement une erreur numérique (bornée à 10^{-3} pour SZ). C'est un compromis inévitable : on accepte une légère dégradation de la précision des résultats (souvent acceptable en physique ou climatologie) en échange d'une réduction potentielle de l'empreinte mémoire. Dans notre cas, ZFP offre le meilleur compromis "Vitesse/Énergie", tandis que SZ serait probablement plus pertinent si le critère principal était le taux de compression pur pour du stockage.

5.4 Recommandations Finales

Sur la base de nos mesures d'efficacité énergétique (Figure 18), nous formulons les recommandations suivantes.

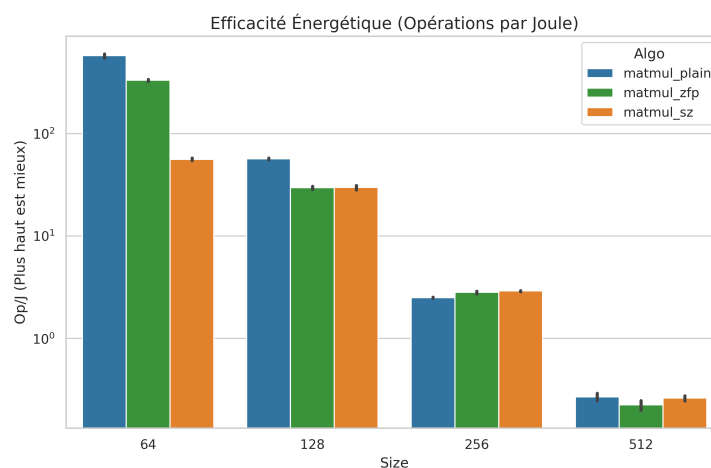


FIGURE 18 – Efficacité énergétique (Opérations par Joule). Plus la barre est haute, plus l'algorithme est efficace.

1. **Éviter la compression pour les petites matrices ($N < 256$) :** Comme le

montre la Figure 18, l'efficacité chute brutalement pour les petites tailles. L'overhead est un gaspillage énergétique pur sans gain réel.

2. **Cibler les goulots d'étranglement mémoire** : La compression devient pertinente uniquement pour les grands problèmes ($N \geq 512$) où elle permet de réduire la pression sur la bande passante mémoire et d'éviter les transferts coûteux (RAM vers Disque).
3. **Privilégier ZFP pour le calcul CPU** : Pour des opérations arithmétiques à la volée, la simplicité algorithmique de ZFP offre un meilleur bilan énergétique que SZ dans nos tests.

Organisation du dossier rendu

Le projet est fourni dans un dossier `Projet_Green_Salma_BENSMAIL` structuré de manière à faciliter la recompilation et la reproductibilité des expériences :

- **Rapport_Green_Computing_Salma_BENSMAIL.pdf** : version finale du présent rapport, décrivant le contexte, la méthodologie expérimentale, les résultats et leur analyse.
- **src/** : codes sources C des noyaux de calcul (`matmul_plain.c`, `matmul_zfp.c`, `matmul_sz.c`, `gauss_*.c`, `cholesky_*.c`), fichier `sz_wrapper.cpp` pour l'intégration de SZ3, ainsi que le `Makefile` permettant de reconstruire les exécutables.
- **include/** : fichiers d'en-tête (`utils.h`, `matmul.h`, `gauss.h`, `cholesky.h`, etc.) rassemblant les prototypes de fonctions et définitions partagées entre les différents modules.
- **data/** : sous-ensemble des jeux de données SDRBench utilisés pour les tests (blocs de température Nyx), permettant de rejouer les scénarios de mesure décrits dans le rapport.
- **scripts/** : scripts d'automatisation `run_final_sdrbench.sh` et `run_experiments.sh`, ainsi que le script d'analyse `analyse_resultats.py` chargé de la normalisation des données et de la génération des figures.
- **results/** : fichiers CSV et graphiques produits par les expériences, notamment `resultats_sdrbench_final.csv` et les figures `graphe_energie.png`, `graphe_temps.png`, `graphe_overhead.png`, `graphe_barres.png` et `graphe_efficacite.png` utilisés dans la section d'analyse.

6 Conclusion

Ce projet de Green Computing nous a permis d'explorer la chaîne complète de l'optimisation énergétique, de l'implémentation bas niveau d'algorithmes scientifiques (Pivot de Gauss, Cholesky) jusqu'à la mesure physique sur matériel réel.

Synthèse des résultats

Nos mesures expérimentales via PowerJoular démontrent que l'utilisation de la compression (ZFP ou SZ) n'est pas neutre énergétiquement. Elle agit comme un investissement :

- **Pour les petites données ($N < 256$)** : L'investissement est à perte. Le surcoût CPU (overhead) lié à la compression/décompression est trop élevé par rapport au temps de calcul utile. La consommation augmente sans gain de performance.
- **Pour les grandes simulations ($N \geq 512$)** : L'investissement devient rentable. Le surcoût énergétique devient marginal (inférieur à 15 %) alors que la réduction de l'empreinte mémoire est significative (facteur 4 à 8).

Comparaison ZFP vs SZ

Dans le cadre spécifique du calcul intensif sur CPU, nos tests mettent en avant **ZFP**. Son algorithme à taux fixe s'est révélé plus rapide et donc plus économe en énergie que l'approche prédictive de SZ, tout en maintenant une précision numérique comparable (erreur $< 10^{-3}$).

Perspectives

Pour aller plus loin, il serait pertinent d'étendre cette étude aux accélérateurs GPU (NVIDIA CUDA). La mémoire étant une ressource critique et limitée sur les cartes graphiques, l'impact positif de la compression y serait probablement encore plus marqué qu'sur CPU.

A Scripts d'Automatisation et d'Analyse

Les scripts présentés ci-dessous garantissent la reproductibilité des mesures effectuées dans ce rapport.

A.1 Script d'automatisation des mesures (Bash)

Ce script pilote l'exécution des binaires, gère les répétitions adaptatives pour les charges faibles et interagit avec l'outil `powerjoular`.

```
1 #!/bin/bash
2 # =====
3 # Script d'automatisation - Projet Green Computing
4 # Auteur : Salma BENSMAIL
5 # Description : Lance les mesures pour Plain, ZFP, SZ
6 # =====
7
8 # Configuration
9 SIZES=(64 128 256 512)
10 ALGOS=("plain" "zfp" "sz")
11 OUTPUT_FILE="../results/resultats_sdrbench_final.csv"
12 NUM_RUNS=5
13
14 # Entete du CSV
15 echo "Algo,Size,Run,Energy_J,Time_s" > $OUTPUT_FILE
16
17 echo "=== Debut de la campagne de mesures ==="
18
19 # Boucle sur les tailles de matrices
20 for N in "${SIZES[@]}"; do
21     echo "Traitement taille N=$N..."
22
23     # Adaptation dynamique des repetitions (Micro-benchmarking)
24     if [ "$N" -eq 64 ]; then
25         REPEATS=10000
26     elif [ "$N" -eq 128 ]; then
27         REPEATS=5000
28     else
29         REPEATS=20
30     fi
31
32     # Boucle sur les algorithmes
33     for ALGO in "${ALGOS[@]}"; do
34         BINARY="../src/matmul_${ALGO}"
35
36         # Verification de l'excutable
```

```
37     if [ ! -f "$BINARY" ]; then echo "Erreur: $BINARY"; continue; fi
38
39     for (( i=1; i<=NUM_RUNS; i++ )); do
40         # Nettoyage cache pour mesure propre
41         sync; echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
42
43         # Lancement PowerJoular en background
44         sudo powerjoular -c -d 0.1 -f /tmp/pj.csv &
45         PJ_PID=$!
46
47         # Execution
48         START=$(date +%s.%N)
49         $BINARY $N $REPEATS > /dev/null
50         END=$(date +%s.%N)
51
52         # Arret mesure
53         sudo kill -SIGINT $PJ_PID
54         wait $PJ_PID 2>/dev/null
55
56         # Calculs
57         DURATION=$(echo "$END - $START" | bc)
58         ENERGY=$(awk -F',' '{sum+=$2} END {print sum}' /tmp/pj.csv)
59
60         echo "$ALGO,$N,$i,$ENERGY,$DURATION" >> $OUTPUT_FILE
61     done
62 done
63 done
```

Listing 1 – run_final_sdrbench.sh

A.2 Script d'Analyse des Données (Python)

Ce script normalise les données brutes (division par le nombre de répétitions) et génère les courbes de tendances avec Seaborn.

```

1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Configuration du style graphique
6 sns.set_theme(style="whitegrid")
7 df = pd.read_csv("../results/resultats_sdrbench_final.csv")
8
9 # --- Normalisation : Ramener a une seule operation ---
10 # Cette fonction doit correspondre aux repetitions du script Bash
11 def get_repeats(n):
12     if n == 64: return 10000
13     if n == 128: return 5000
14     return 20
15
16 # Application de la normalisation
17 df['Repeats'] = df['Size'].apply(get_repeats)
18 df['Time_Unit'] = df['Time_s'] / df['Repeats']
19 df['Energy_Unit'] = df['Energy_J'] / df['Repeats']
20
21 # --- Generation des Graphiques ---
22 # 1. Consommation Energetique
23 plt.figure(figsize=(10, 6))
24 sns.lineplot(data=df, x='Size', y='Energy_Unit', hue='Algo', marker='o')
25 plt.yscale('log')
26 plt.title('Consommation Energetique Unitaire (Log)')
27 plt.ylabel('Energie (Joules / Op)')
28 plt.xlabel('Taille de Matrice (N)')
29 plt.savefig('../results/graphe_energie.png')
30 plt.close()
31 # 2. Performance Temporelle
32 plt.figure(figsize=(10, 6))
33 sns.lineplot(data=df, x='Size', y='Time_Unit', hue='Algo', marker='s')
34 plt.yscale('log')
35 plt.title('Temps d\'Execution Unitaire (Log)')
36 plt.ylabel('Temps (Secondes / Op)')
37 plt.xlabel('Taille de Matrice (N)')
38 plt.savefig('../results/graphe_temps.png')
39 plt.close()
40 print("Graphiques generes avec succes dans /results !")

```

Listing 2 – analyse_resultats.py